RAMACHANDRA COLLEGE OF ENGINEERING AUTONOMOUS

Approved by AICTE, New Delhi
Permanently Affiliated to JNTUK
Recognized by UGC 2(f) & 12(B)
Accredited by NAAC A+ &
NBA (UG - CE, EEE, ME, ECE & CSE)
ISO 9001: 2015 Certified
NH-16 Bypass Road, Eluru-534007, A.P.

Department of Electronics and Communication Engineering

LANDING PAGE

(Mini Project)

Team Members:

1. M.N.V. RAGHU RAM (24ME1A04H4)
2. B. KARTHIK (25ME5A0415)
3. S. MAHESH (24ME1A04I7)
4. V.RAMU (24ME1A04J8)
5. P.PRUDHVI (24ME1A04H7)

Under the Guidance of :

Mr. P . Pavan kumar ,Trainer

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
RAMACHANDRA COLLEGE OF ENGINEERING

(AUTONOMOUS)

(APPROVED BY AICTE - ACCREDITED BY NBA)

ELURU – 534007

INDEX

S.NO	Contents	Page.No
1	Abstract	3
2	Introduction	4-5
3	Technologies Involved	6-7
4	Key Features	8-9
5	Project Setup	10-11
6	Design Philosophy	12-13
7	Code	14-22
8	Action	23-24
9	Home Page	25-29
10	Website UI Breakdown	30-31
11	Future Enhancements	32-33
12	Conclusion	34
13	Reference	35

LANDING PAGE

ABSTRACT

In today's competitive job market, creating a well-structured and professional resume is very important for job seekers. Many candidates face difficulties in identifying weaknesses in their resumes and improving the content to meet industry standards. The proposed project, **AI Resume Analyzer and Improver**, aims to solve this problem by using Generative Artificial Intelligence to automatically evaluate and enhance resumes. The system allows users to upload their resume and receive intelligent feedback on the quality and effectiveness of their resume content.

The system is developed using the Python programming language and provides a user-friendly interface for uploading resumes. The application extracts text from the uploaded resume and processes it using a Generative AI model. The AI analyzes the resume based on various parameters such as clarity, structure, skills representation, and professional language. Based on this analysis, the system generates a resume score, identifies weak areas, and provides suggestions for improvement.

Additionally, the system improves the quality of resume content by rewriting weak bullet points in a more professional and impactful manner. This helps users present their skills and achievements more effectively. The proposed solution demonstrates how Generative AI can assist individuals in improving their resumes and increasing their chances of success in job applications. The project highlights the practical application of AI in career development and intelligent document analysis.

INTRODUCTION

In the present digital world, job opportunities are increasing rapidly, and competition among job seekers is also growing significantly. A resume plays a crucial role in presenting a candidate's qualifications, skills, achievements, and experience to potential employers. It serves as the first impression that recruiters have about a candidate. Therefore, having a clear, well-structured, and professional resume is very important for increasing the chances of getting shortlisted for job interviews. However, many students and fresh graduates find it difficult to prepare an effective resume because they may not know the proper format, industry standards, or the best way to describe their skills and achievements.

In recent years, Artificial Intelligence has emerged as a powerful technology that can analyze large amounts of textual data and generate meaningful insights. Generative AI models are capable of understanding natural language and producing human-like responses. These models can be used in various applications such as text generation, content improvement, document analysis, and automated feedback systems. By applying Generative AI techniques, it is possible to analyze resume content automatically and provide useful suggestions to improve the quality, structure, and clarity of the document.

The proposed project, AI Resume Analyzer and Improver, is developed using the Python programming language and utilizes Generative AI to analyze and enhance resumes. The system allows users to upload their resume in digital format, after which the application extracts the textual content from the resume. The extracted information is then analyzed by the AI model to evaluate the overall quality of the resume. Based on this analysis, the system generates a resume score, identifies weak areas, and provides suggestions for improvement. Additionally, the system rewrites weak bullet points in a more professional and impactful way. This project demonstrates how Generative AI can be used as an intelligent tool to help individuals create better resumes and improve their chances of success in job applications.

TECHNOLOGIES INVOLVED

Python:
Python is the main programming language used to develop this project. It is easy to understand and widely used for building applications related to data processing and automation. In this project, Python is used to implement the core functionality such as extracting text from the resume, analyzing the content, calculating the resume score, generating suggestions, and rewriting bullet points. Python also provides many useful libraries that make the development process faster and easier.

Streamlit:
Streamlit is a Python framework used to build web applications quickly and easily. It allows developers to create interactive user interfaces with very little code. In this project, Streamlit is used to design the front-end interface where users can upload their resumes and view the analysis results. It also helps in displaying the resume score, improvement suggestions, and improved bullet points in a clear and user-friendly format.

PyPDF2:
PyPDF2 is a Python library used to read and extract text from PDF files. Since most resumes are shared in PDF format, this library is used to access the content of the uploaded resume. The extracted text is then processed and analyzed by the system to check for important sections like education, skills, projects, and experience.

Visual Studio Code (VS Code):
Visual Studio Code is the code editor used to develop this project. It provides features like syntax highlighting, debugging support, and extensions that help developers write and manage code efficiently. VS Code makes the development process easier and more organized.

PDF File Format:
The application accepts resumes in PDF format because it is one of the most commonly used formats for professional documents. PDF files maintain proper formatting and are easy to share. The system reads the uploaded PDF file and analyzes the text content to evaluate the quality of the resume.

KEY FEATURES

The application allows users to upload their resumes in PDF format through a simple and user-friendly interface. The uploaded file is then processed by the system for analysis.

Automatic Resume Analysis:
After uploading the resume, the system automatically extracts the text from the PDF file and analyzes the content. It checks whether important sections like education, skills, projects, internships, and experience are present in the resume.

Resume Scoring System:
The system calculates a resume score based on the presence of important keywords and the length of the resume. This score helps users understand how well their resume is structured and how complete it is.

Improvement Suggestions:
The application provides useful suggestions to improve the resume. For example, it may suggest adding a career objective, including projects, mentioning technical skills clearly, or increasing the resume content if it is too short.

Bullet Point Improvement:
The system also improves some lines from the resume by rewriting them as better and more professional bullet points. This helps users present their work and achievements more effectively.

Simple and Interactive Interface:
The application is designed using Streamlit, which provides a clean and easy-to-use interface. Users can upload their resumes and instantly view the analysis results without any complicated steps.

Quick Results:
The system analyzes the resume quickly and displays the score, suggestions, and improved bullet points within a few seconds, making the process fast and efficient.

PROJECT SETUP

The project setup is an important step in developing the Resume Analyzer & Improver application. It involves installing the required software, organizing project files, and preparing the environment to run the application smoothly. A proper setup helps in managing the project efficiently and ensures that the system works correctly.

1. Folder Structure Creation
The first step is to create a proper folder structure for the project. This helps in organizing the files properlyand makes the project easier to manage.
Example structure of the project:

```
Resume Analyzer Project/
│
├── app.py → Main Python file containing the application code
├── requirements.txt → List of required Python libraries (optional)
├── resumes/ → Folder to store sample resumes for testing
└── README.md → Project description and information (optional)
```

2. Installing Python
Python must be installed on the system to run the project. Python is the main programming language used to develop the Resume Analyzer & Improver application. After installation, the user can verify it by running the python command in the terminal.

3. Installing Required Libraries
The project requires certain Python libraries to function properly. These libraries help in building the interface and extracting text from PDF files.
Required libraries include:

- Streamlit – Used to create the web application interface
- PyPDF2 – Used to extract text from PDF resumes

They can be installed using the following commands:

```
pip install streamlit
pip install PyPDF2
```

4. Setting Up the Development Environment
A code editor such as Visual Studio Code is used to write and manage the project code. VS Code provides useful features like syntax highlighting, debugging support, and extensions that make coding easier and more efficient.

5. Creating the Main Application File
The main project code is written inside a Python file called **app.py**. This file contains the logic for uploading resumes, extracting text, calculating the resume score, generating suggestions, and displaying the results.

6. Running the Application
After completing the code, the application can be executed using the following command in the terminal:

```
streamlit run app.py
```

This command starts the Streamlit server and automatically opens the application in a web browser.

7. Testing the Application
After running the application, a user can upload a resume in PDF format through the interface. The system will then analyze the resume and display the score, suggestions, and improved bullet points. This step ensures that the application works correctly.

DESIGN PHILOSOPHY

The design philosophy of the Resume Analyzer & Improver project focuses on **simplicity, usability, and efficiency**. The goal is to make it easy for users, especially students and job seekers, to analyze and improve their resumes without any technical knowledge.

Key Points of the Design Philosophy:

User-Friendly Interface:
The application is designed with Streamlit to provide a clean and simple interface. Users can upload their resumes and immediately see the analysis results without navigating through complex menus.

Minimal Steps:
The design ensures that only a few steps are needed: upload the resume, view the score, get suggestions, and see improved bullet points. This reduces confusion and saves time.

Automation and Efficiency:
The system automatically extracts text from the PDF, analyzes it, calculates a score, and generates suggestions. Users do not need to manually check for errors or missing sections.

Focus on Key Resume Elements:
The design prioritizes important resume components such as education, skills, projects, internships, and experience. This ensures that users get meaningful feedback.

Modular Structure:
The code is structured in modules for text extraction, scoring, suggestions, and bullet improvement. This makes it easy to maintain and extend the system in the future.

Quick Feedback:
The application provides instant results, which helps users make improvements immediately. This real-time feedback is central to the design philosophy.

CODE

```
import streamlit as st
import PyPDF2


# ---------- Extract text from PDF ----------
def extract_text_from_pdf(pdf_file):
    reader = PyPDF2.PdfReader(pdf_file)
    text = ""

    for page in reader.pages:
        if page.extract_text():
            text += page.extract_text()

    return text


# ---------- Calculate Resume Score ----------
def calculate_score(text):
    score = 0
    text = text.lower()

    keywords = ["education", "skills", "project", "internship", "experience"]

    for word in keywords:
        if word in text:
            score += 20

    if len(text) > 1000:
        score += 10

    return min(score, 100)
```

```
# ---------- Suggestions ----------
def suggest_improvements(text):
    suggestions = []
    text = text.lower()

    if "objective" not in text:
        suggestions.append("Add career objective section")

    if "project" not in text:
        suggestions.append("Add at least one project")

    if "skills" not in text:
        suggestions.append("Mention technical skills clearly")

    if len(text) < 800:
        suggestions.append("Resume content is too short")

    return suggestions

# ---------- Rewrite bullet points ----------
def rewrite_bullets(text):
    lines = text.split("\n")
    improved = []

    for line in lines:
        if len(line.strip()) > 20:
            improved.append("✓ Successfully worked on: " + line.strip())

    return improved[:5]

# ---------- STREAMLIT UI ----------

st.title("Resume Analyzer & Improver")

uploaded_file = st.file_uploader("Upload your Resume (PDF)", type=["pdf"])
```

```
if uploaded_file is not None:

    text = extract_text_from_pdf(uploaded_file)

    st.subheader("Resume Score")
    score = calculate_score(text)
    st.write(score, "/ 100")

    st.subheader("Suggestions")

    suggestions = suggest_improvements(text)

    if suggestions:
        for s in suggestions:
            st.write("•", s)
    else:
        st.write("Your resume looks good!")

    st.subheader("Improved Bullet Points")

    bullets = rewrite_bullets(text)

    for b in bullets:
        st.write(b)


cd OneDrive\Desktop\GEN_AI_4H4\Resume.py
python -m streamlit run resumecode.py
python -m streamlit run resumecode.py
```

ACTIONS

Key Actions of the System:

Resume Upload:

- The user uploads a resume in PDF format through the web interface.
- The system accepts only PDF files to ensure proper text extraction.

Text Extraction:

- The system reads the uploaded PDF using the PyPDF2 library.
- All text from the resume is extracted for further analysis.

Resume Analysis:

- The extracted text is scanned for important keywords like education, skills, projects, internships, and experience.
- The system evaluates the presence of these sections to determine the completeness of the resume.

Resume Scoring:

- A score is calculated based on keyword presence and text length.
- The score helps the user understand the overall quality of the resume.

Suggestion Generation:

- If the resume is missing important sections, the system generates suggestions for improvement.
- Suggestions include adding a career objective, mentioning skills clearly, or including projects.

Bullet Point Improvement:

- The system identifies long or important lines in the resume.
- These lines are rewritten into clear, professional bullet points to improve readability and impact.

Result Display:

- The resume score, suggestions, and improved bullet points are displayed on the Streamlit interface.
- Users can immediately see the analysis results and take action to improve their resumes.

HOME PAGE

The **Home Page** of the Resume Analyzer & Improver is the main interface where users interact with the application. It is designed to be **simple, clean, and easy to use**, allowing users to upload resumes and receive feedback quickly without any technical knowledge.

Features of the Home Page:

Title and Instructions:

- The page displays a clear and visible title, such as *"Resume Analyzer & Improver"*.
- A short description or instruction is provided to guide users on how to upload their resume. This ensures that even first-time users can understand the process easily.

Resume Upload Section:

- Users can click the upload button to select a PDF file from their computer.
- The system only accepts PDF files to ensure proper text extraction and analysis.
- A clear message or alert is shown if the user tries to upload an unsupported file type.

Display Analysis Results:

- After uploading a resume, the home page displays the analysis results directly:
  - **Resume Score** – A number out of 100 indicating the overall quality and completeness of the resume.
  - **Suggestions** – Personalized tips for improving missing or weak sections such as education, skills, or projects.
  - **Improved Bullet Points** – Key points from the resume rewritten in a professional and clear manner to highlight achievements and experience.

Interactive and Real-Time Feedback:

- The home page provides real-time results as soon as the resume is uploaded.
-

Users can see their resume score, suggestions, and improved bullet points instantly without navigating to other pages.

User-Friendly Design:

- The interface is designed to be minimalistic and intuitive.
- Important elements like upload button, score, suggestions, and improved bullet points are displayed clearly to avoid confusion.

Encouraging Feedback:

- The system provides constructive suggestions and positive reinforcement.
- Users are motivated to improve their resumes based on actionable feedback, making the home page not only functional but also encouraging.

WEBSITE UI BREAKDOWN

1 The **Website UI** of the Resume Analyzer & Improver is designed to be **simple, clean, and interactive**, ensuring that users can easily upload resumes and view results without confusion. The UI is divided into several sections, each serving a specific purpose.

1. Header Section:

- Displays the title of the application: *"Resume Analyzer & Improver"*.
- Provides a brief description or instruction to guide the user on uploading the resume.
- The header is styled to be clear and visually appealing so that users immediately understand the purpose of the page.

2. Resume Upload Section:

- Contains a file uploader where users can select their resume in PDF format.
- The uploader ensures only PDF files are accepted to prevent errors in text extraction.
- Clear instructions are provided to help users understand how to upload the file.

3. Resume Score Display:

- After uploading the resume, the score is displayed prominently.

- The score is out of 100 and indicates the overall quality and completeness of the resume.
- This section uses visual cues (like bold text or color highlights) to make the score easily noticeable.

4. Suggestions Section:

- Lists actionable suggestions for improving the resume.
- For example, it may recommend adding a career objective, listing technical skills clearly, or including projects.
- Suggestions are presented in bullet points for easy readability.

5. Improved Bullet Points Section:

- Shows rewritten bullet points from the user's resume to make them sound more professional and impactful.
- Only the most relevant points are displayed to avoid clutter.
- Each bullet starts with a positive action word (like "✓ Successfully…") to enhance clarity and professionalism.

6. Layout and Design:

- The UI uses a clean layout with clearly separated sections.
- Minimal colors and clear fonts are used to maintain readability.
- Streamlit handles the responsiveness, so the application works on different screen sizes without issues.

7. Real-Time Feedback:

- All analysis results (score, suggestions, improved bullets) are displayed immediately after the resume is uploaded.
- Users do not need to navigate to another page, making the experience fast and interactive.

FUTURE ENHANCEMENT

upport for Multiple File Formats:

- Currently, the system accepts only PDF resumes.
- Future versions can support Word documents (.docx) and text files (.txt) to allow more flexibility for users.

Advanced AI-Based Analysis:

- Use Natural Language Processing (NLP) and AI models to analyze the resume more accurately.
- The system can detect grammar issues, readability, and better evaluate achievements and experience.

Detailed Skill Evaluation:

- Include a feature to evaluate technical and soft skills separately.
- Suggest improvements based on the skills listed and industry requirements.

Formatting and Design Suggestions:

- Provide suggestions for resume layout, font size, spacing, and overall presentation.
- Help users create resumes that look visually professional in addition to having strong content.

Interactive Dashboard:

- Allow users to track improvements over time by uploading multiple versions of their resumes.
- Show progress in skills, projects, and experience through visual charts.

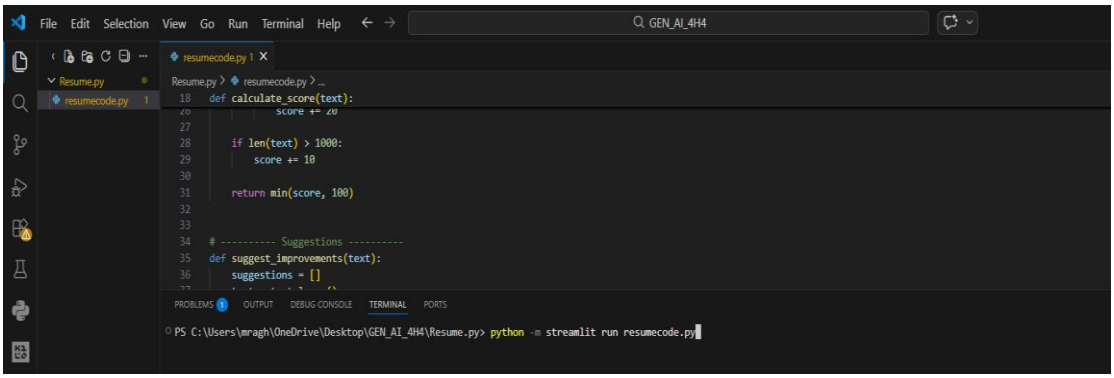
Integration with Job Portals:

- Enable users to directly share or submit their improved resumes to job portals or email from the application.

Multi-Language Support:

- Include the option to analyze resumes in different languages to cater to a wider audience.

Results



1. Project Development Environment
This project was developed using **Python programming language** in the **Visual Studio Code (VS Code)** development environment. Required libraries such as **Streamlit** and **PyPDF2** were installed to build the application. Streamlit was used to create a simple web interface where users can upload their resumes, and PyPDF2 was used to extract text content from the uploaded PDF files.

2. Writing the Program in VS Code
The project code was written in a Python file named **resumecode.py**. The program includes several functions that perform different tasks. The first function extracts text from the uploaded resume file using the PyPDF2 library. The second function analyzes the extracted text and calculates a resume score based on the presence of important keywords such as education, skills, projects, internship, and experience. Additional functions were implemented to suggest improvements and rewrite bullet points in a more professional manner.

3. Running the Application
After writing the code, the application was executed in the terminal using the Streamlit command. The command used to run the application is:
python -m streamlit run resumecode.py
Once the command is executed, Streamlit automatically starts a local web server and generates a link (localhost). This link opens the application in the browser, where the user can interact with the Resume Analyzer system.

4. Uploading the Resume
The user uploads a resume in **PDF format** through the upload interface provided on the web page. Once the resume is uploaded, the system reads the file and extracts all the textual content from the PDF document. The extracted text becomes the input for the analysis process.

---

5. Resume Analysis Process
After extracting the text, the program analyzes the resume by checking for important sections such as education, skills, projects, internship, and experience. Each detected keyword increases the overall resume score. The system also evaluates the length of the resume to determine whether sufficient information is provided. Based on this analysis, a resume score out of 100 is calculated.

6. Generating Suggestions
The application then generates improvement suggestions based on the missing sections in the resume. For example, if the resume does not contain a career objective, project information, or technical skills, the system recommends adding these sections. These suggestions help users enhance the quality of their resumes and make them more suitable for job applications.

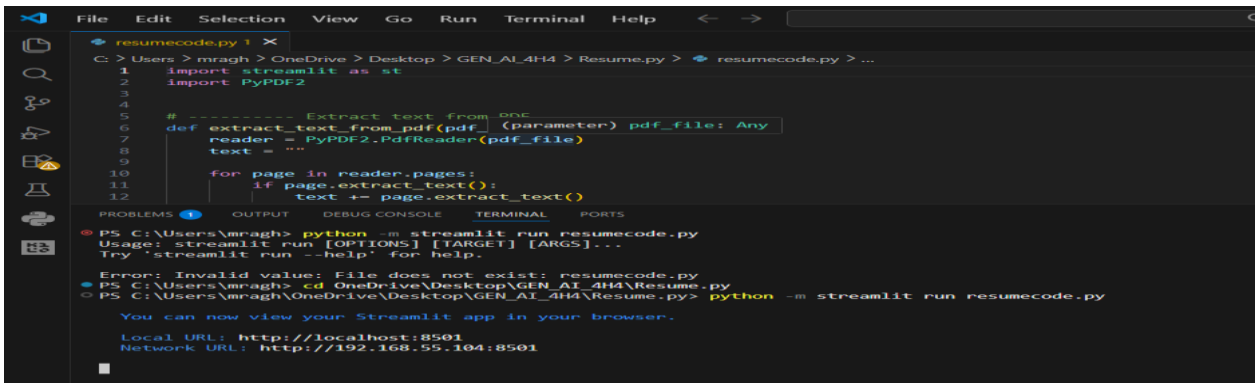
7. Improving Bullet Points
The system also rewrites certain lines from the resume into more professional bullet points. This feature improves the clarity and impact of the resume by presenting information in a structured and achievement-oriented format.

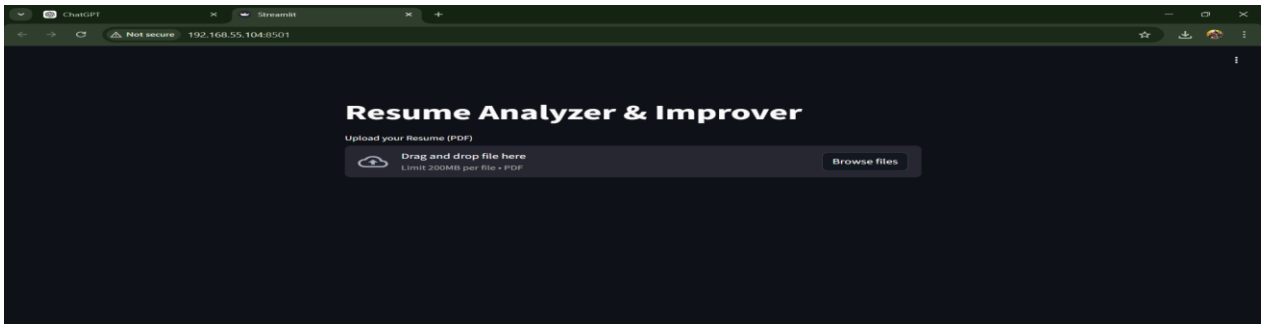
8. Displaying the Results
Finally, the system displays the results on the web interface. The output includes:

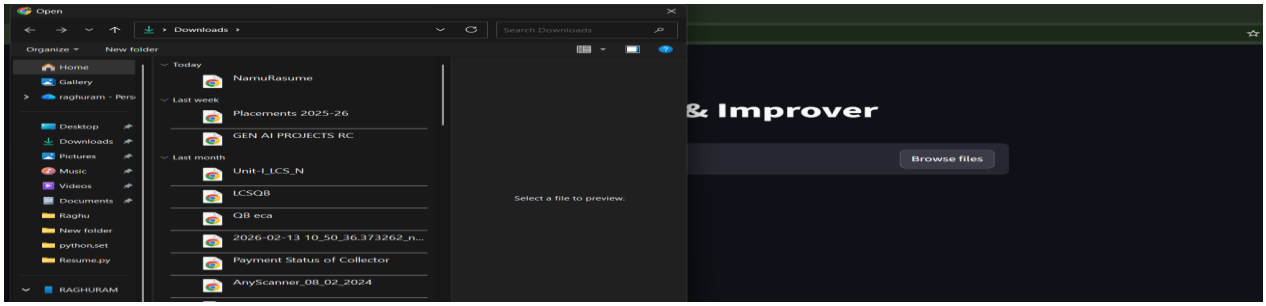
- Resume Score
- Suggested Improvements
- Improved Bullet Points

This allows users to quickly understand the strengths and weaknesses of their resumes and make necessary modifications.

9. Final Outcome
The **Resume Analyzer & Improver** application successfully analyzes uploaded resumes and provides meaningful feedback to users. It helps students and job seekers improve their resumes by identifying missing sections, enhancing bullet points, and providing an overall resume score. The project demonstrates how artificial intelligence-based analysis can assist users in creating stronger and more professional resumes.







Resume Score

70 / 100

Suggestions

Your resume looks good!

Improved Bullet Points

✓ Successfully worked on: Email: namrathasaidornala66@gmail.com @gmail.com

✓ Successfully worked on: Seeking a position to enhance my skill and technical knowledge and help in all round growth of both

✓ Successfully worked on: technical as well as man agerial intellect.

✓ Successfully worked on: ACADEMIC QUALIFICATION:

✓ Successfully worked on: COURSE SCHOOL/COLLEGE BOARD ACADEMIC

Conclusion

The **Resume Analyzer & Improver** project was successfully developed to help users evaluate and improve their resumes using artificial intelligence techniques. The system allows users to upload their resumes in PDF format and automatically analyzes the content to identify important sections such as education, skills, projects, internship, and experience. Based on this analysis, the application calculates a resume score and provides useful suggestions for improvement.

The project also helps users enhance their resumes by rewriting certain bullet points in a more professional and structured format. The application was implemented using Python with the help of **Streamlit** for creating the user interface and **PyPDF2** for extracting text from PDF documents. These technologies enabled the development of a simple, interactive, and user-friendly web application.

Overall, the project demonstrates how artificial intelligence and natural language processing techniques can be used to assist job seekers in improving the quality of their resumes. By providing instant feedback and recommendations, the system helps users create more effective resumes that increase their chances of securing job opportunities. This project also highlights the practical application of AI in real-world career development tools.

CONCLUSION

The **Resume Analyzer & Improver** project was successfully developed to help users evaluate and improve their resumes using artificial intelligence techniques. The system allows users to upload their resumes in PDF format and automatically analyzes the content to identify important sections such as education, skills, projects, internship, and experience. Based on this analysis, the application calculates a resume score and provides useful suggestions for improvement.

The project also helps users enhance their resumes by rewriting certain bullet points in a more professional and structured format. The application was implemented using Python with the help of **Streamlit** for creating the user interface and **PyPDF2** for extracting text from PDF documents. These technologies enabled the development of a simple, interactive, and user-friendly web application.

Overall, the project demonstrates how artificial intelligence and natural language processing techniques can be used to assist job seekers in improving the quality of their resumes. By providing instant feedback and recommendations, the system helps users create more effective resumes that increase their chances of securing job opportunities. This project also highlights the practical application of AI in real-world career development tools.

REFERENCE

1. 1.Python Official Documentation – Comprehensive reference for Python programming language, standard libraries, and syntax.
   - https://docs.python.org/3/
2. Streamlit Official Documentation – Guides for creating interactive web applications with Python.
   - https://docs.streamlit.io/
3. PyPDF2 Library Documentation – For reading, writing, and extracting text from PDF files.
   - https://pythonhosted.org/PyPDF2/
4. Real Python Tutorials – Tutorials on Python programming, file handling, and automation.
   - https://realpython.com/
5. GeeksforGeeks Python Tutorials – Detailed guides on Python libraries, file processing, and project examples.
   - https://www.geeksforgeeks.org/python-programming-language/
6. Medium Articles on Resume Analysis using Python – Blog posts covering resume parsing, scoring, and automation techniques.
   - https://medium.com/tag/python
7. Kaggle Datasets & Projects – Sample resume datasets and Python-based projects for text extraction and NLP.
   - https://www.kaggle.com/
8. GitHub Open-Source Projects – Example repositories for resume analyzers, AI-based scoring systems, and Streamlit dashboards.
   - https://github.com/
9. W3Schools Python Tutorials – Python basics, file handling, and library usage references.
   - https://www.w3schools.com/python/
10. Towards Data Science – NLP in Resume Parsing – Articles explaining how NLP can analyze resumes and extract key information.
    - https://towardsdatascience.com/
11. Stack Overflow – Problem-solving and coding examples for Python, Streamlit, and PDF processing issues.
    - https://stackoverflow.com/
12. Python Projects for Beginners – YouTube Tutorials – Video tutorials for developing small Python projects including resume analyzers.
    - https://www.youtube.com/
13. OpenCV & PDF Text Extraction Tutorials – For enhancing resume parsing and processing techniques.
    - https://opencv.org/
14. Research Papers on Resume Analysis and NLP Techniques – Academic references for AI-based resume evaluation.
    - "Automatic Resume Screening using NLP and Machine Learning" – IEEE Xplore

- **“Resume Parsing and Skill Extraction using NLP” – ResearchGate**

15. **College Project Report Guidelines – References on formatting, headings, and proper documentation standards.**
16. **Python Package Index (PyPI) – Official repository for Python packages like PyPDF2 and other libraries.**
    - **https://pypi.org/**
17. **Streamlit Community Forum – For discussions, troubleshooting, and learning best practices in Streamlit.**
    - **https://discuss.streamlit.io/**
18. **Python Crash Course Book (Eric Matthes) – Reference book for Python project development and best practices.**
19. **Python for Data Analysis (Wes McKinney) – Useful reference for text data handling and analysis techniques.**
20. **Medium Articles on Streamlit Dashboard Projects – Step-by-step guides to create interactive dashboards for projects.**